FOML-AI2000 -Hackathon

Report

Team: ByteBots

Team Mates: Challa Srikrishna Reddy – ES22BTECH11006

R Kiran Kumar – ES22BTECH11030

Report by: R Kiran Kumar

Key points:

- **Model** : XGBoost Algorithm with Threshold Optimization.
- Peak F1-score observed: 0.441

Detailed Procedure & observations along with individual work:

Data Preprocessing:

- Started by separating the target variables and assigning numerical values (0, 1, 2) to each label.
- Analyzed the percentage distribution of each label in the training data and observed a significant bias toward the medium class (60%), indicating an imbalance in the dataset.
- Noted that the data contains many missing values and high cardinality in certain columns, identifying these as categorical features.
- Dropped features with more than 80% missing values, as they had little influence on prediction and could introduce noise if filled with other values.
- After examining the dataset, removed features like IDs and redundant columns.
- Reviewed central tendency measures (mean, median, mode) to fill NaN values with median or mode, as shown in the notebook.

- Examined other features to assess their importance and reliability.

My work here: Dropped features with high missing values, removed irrelevant columns, filled NaN values with median or mode.

.

Model Selection:

- Tested the model on sklearn's Random Forest Classifier and observed that scores were low, with very low prediction accuracy in the low (0) and high (2) labels due to data imbalance.
- Hence, a fine-tuning process was followed to improve scores without overfitting the model.
- As part of fine-tuning, used **threshold optimization** to help reduce imbalance and enhance recall and precision abilities.
- Instead of directly predicting labels, predicted class probabilities and imposed specific threshold values for the low and high classes, thereby increasing their likelihood in predictions.
- Threshold values were initially set based on observation, then refined through a search over a limited range to achieve the best F1 score without overfitting the model.
- Explored various algorithms to improve model performance, ultimately selecting the XGBoost algorithm as the best fit.
- Please install using **!pip install xgboost** (also added in requirements.txt).
- This approach yielded the best possible results from our efforts.

My Work Here: Tested the Random Forest Classifier and observed accuracy issues, especially in the low (0) and high (2) labels, due to data imbalance. To address this, I implemented a fine-tuning process, adjusting threshold values for class probabilities to enhance recall and precision. I optimized these thresholds by searching for values that improved the F1 score without overfitting.

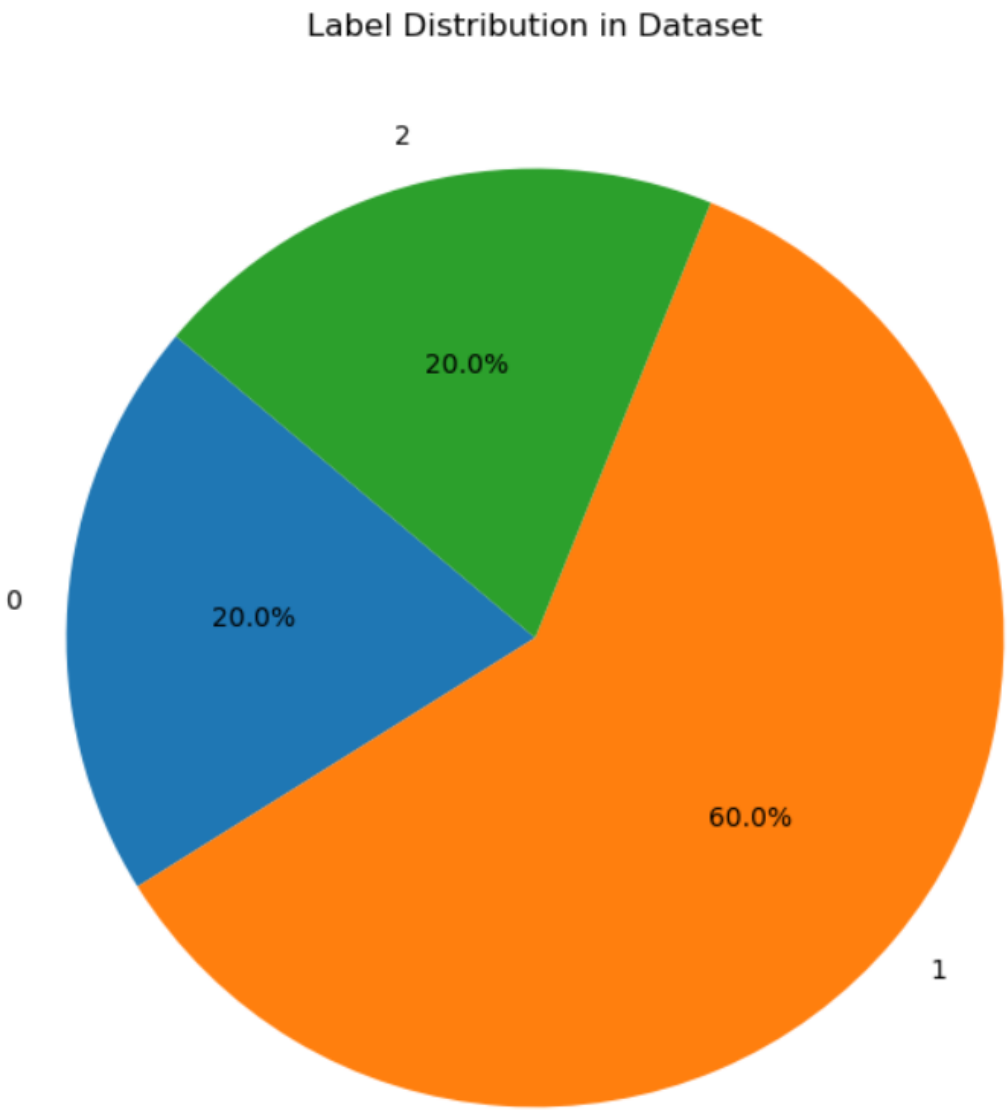
Model F-1 Stats(Average):

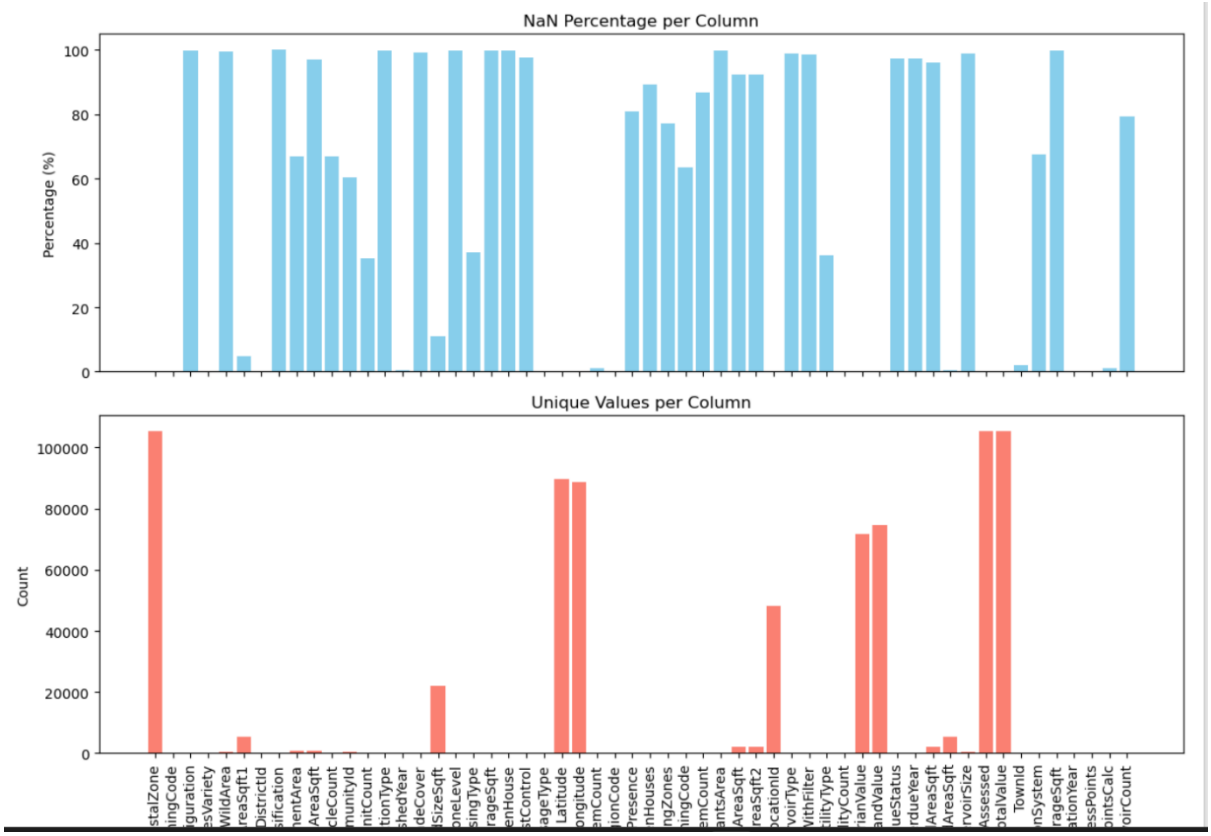
Random Forest: 0.354

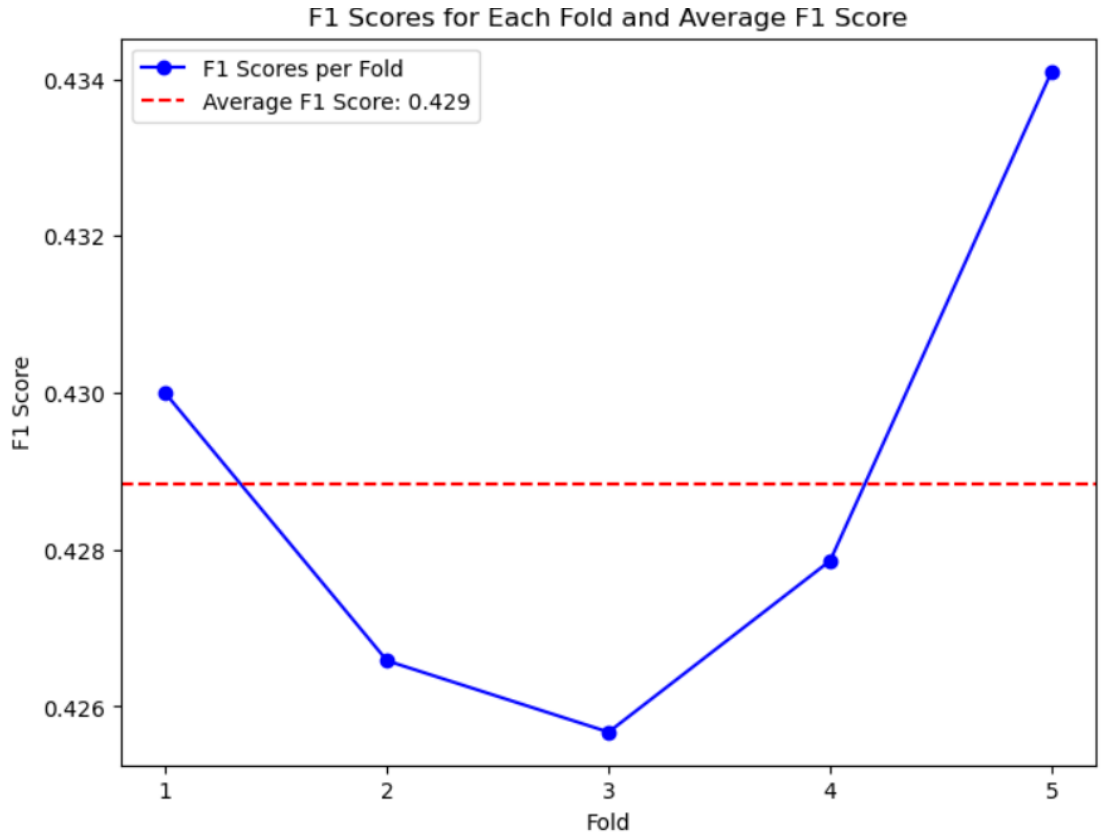
Random Forest with Fine Tuning(Threshold optimization) : 0.419

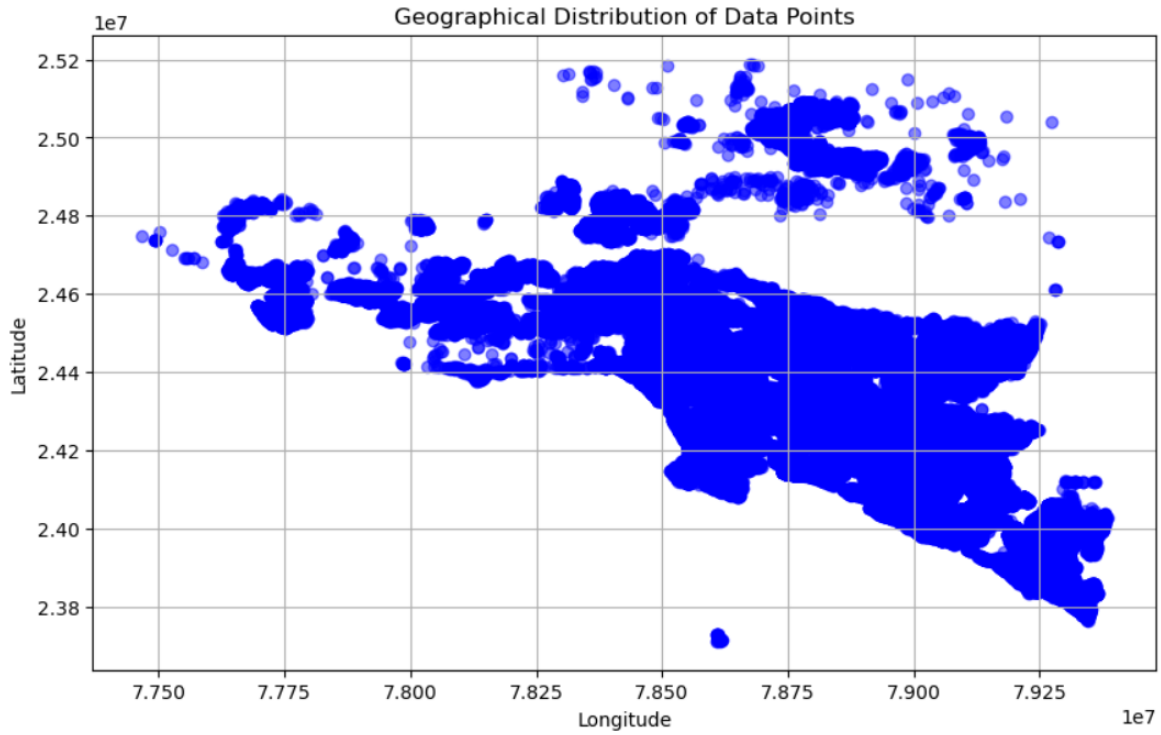
XGBoost with Fine Tuning: 0.436

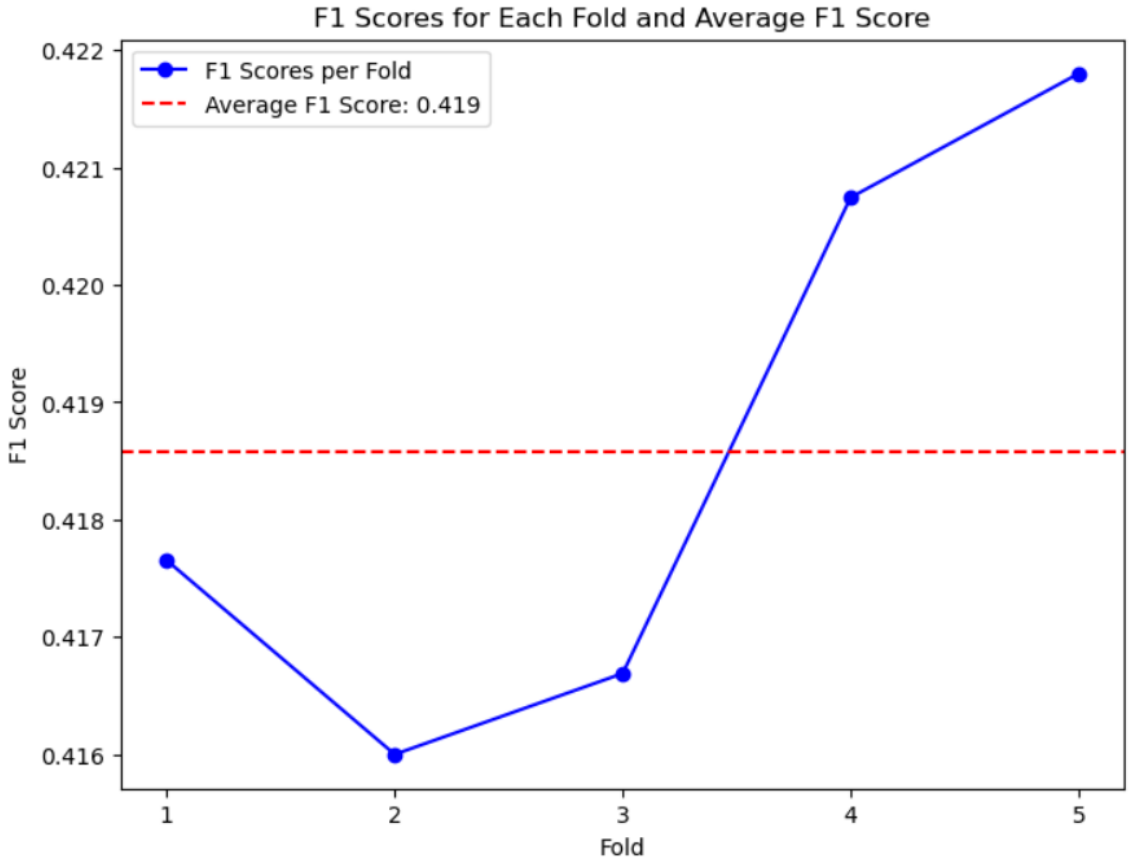
Note: as part of preparing the notebook as per given criterion, lot of plots and relevant data are deleted..please find few of the plots in the notebook.


Label Distribution in Dataset
2
20.0%
0
20.0%
60.0%
1


NaN Percentage per Column
Percentage (%)
Unique Values per Column
Count


F1 Scores for Each Fold and Average F1 Score
F1 Scores per Fold
Average F1 Score: 0.429
F1 Score
Fold


Geographical Distribution of Data Points
1e7
2.52
2.50
2.48
2.46
2.44
2.42
2.40
2.38
Latitude
7.750
7.775
7.800
7.825
7.850
7.875
7.900
7.925
Longitude
1e7


F1 Scores for Each Fold and Average F1 Score
F1 Scores per Fold
Average F1 Score: 0.419
0.422
0.421
0.420
0.419
0.418
0.417
0.416
F1 Score
1
2
3
4
5
Fold